

FP-Growth (Frequent Pattern Growth) Algorithm

FP-Growth is an association rule mining algorithm used to find **frequent itemsets** in large datasets.

It was developed as an improvement over the **Apriori Algorithm** because Apriori generates a huge number of candidate itemsets and requires multiple database scans.

Main Idea

Instead of generating candidate itemsets, FP-Growth:

1. Scans the database to find frequent items.
2. Compresses the database into an **FP-Tree (Frequent Pattern Tree)**.
3. Mines frequent patterns directly from the FP-Tree.

As a result, FP-Growth is typically much faster than Apriori for large datasets.

Example Dataset

Suppose a supermarket has the following transactions:

Transaction ID	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer, Eggs
T3	Milk, Diaper, Beer, Cola
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper, Cola

Assume:

Minimum Support = 3

Step 1: Count Item Frequencies

Item Frequency

Bread 4
Milk 4
Diaper 4

Item Frequency

Beer 3

Cola 2

Eggs 1

Remove infrequent items:

Cola (2) ✖

Eggs (1) ✖

Remaining items:

Bread (4)

Milk (4)

Diaper (4)

Beer (3)

Step 2: Sort Items by Frequency

A common order is:

Bread → Milk → Diaper → Beer

Rearrange transactions according to this order.

Transaction	Ordered Items
T1	Bread, Milk
T2	Bread, Diaper, Beer
T3	Milk, Diaper, Beer
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper

Step 3: Build FP-Tree

Start with a root node.

Insert T1

```
Root
└─ Bread:1
   └─ Milk:1
```

Insert T2

```
Root
└─ Bread:2
   └─ Milk:1
      └─ Diaper:1
         └─ Beer:1
```

Insert T3

```
Root
├─ Bread:2
│  └─ Milk:1
│     └─ Diaper:1
│        └─ Beer:1
├─ Milk:1
│  └─ Diaper:1
│     └─ Beer:1
```

Insert Remaining Transactions

Final FP-Tree:

```
Root
├─ Bread:4
│  └─ Milk:3
│     └─ Diaper:2
│        └─ Beer:1
│  └─ Diaper:1
│     └─ Beer:1
├─ Milk:1
│  └─ Diaper:1
│     └─ Beer:1
```

Notice how common prefixes are shared, compressing the database.

Step 4: Mine Frequent Patterns

Start from the least frequent item:

Beer

Find all paths leading to Beer:

```
Bread → Diaper → Beer
Milk → Diaper → Beer
Bread → Milk → Diaper → Beer
```

Generate frequent itemsets such as:

```
{Beer}
{Diaper, Beer}
{Bread, Beer}
{Milk, Beer}
{Bread, Diaper, Beer}
```

Repeat for all items.

Frequent Itemsets Found

Examples:

Itemset	Support
{Bread}	4
{Milk}	4
{Diaper}	4
{Beer}	3
{Bread, Milk}	3
{Milk, Diaper}	3
{Diaper, Beer}	3

Generating Association Rules

From:

```
{Diaper, Beer}
```

Support = 3

Possible rule:

```
Diaper → Beer
```

Confidence:

$$\text{Confidence} = \frac{\text{Support}(\text{Diaper, Beer})}{\text{Support}(\text{Diaper})}$$

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$$

$$= \frac{3}{4} = 75\%$$

Interpretation:

75% of customers who buy diapers also buy beer.

FP-Growth vs Apriori

Feature	Apriori	FP-Growth
Candidate Generation	Yes	No
Database Scans	Multiple	Usually 2
Speed	Slower	Faster
Memory Usage	Higher	Lower
Large Datasets	Less Efficient	More Efficient

Why FP-Growth Is Faster

Apriori

```
Generate Candidates
↓
Scan Database
↓
Generate More Candidates
↓
Scan Again
↓
Repeat
```

Many scans and many candidate itemsets.

FP-Growth

```
Scan Database Twice
↓
Build FP-Tree
↓
Mine Patterns Directly
```

No candidate generation.

Interview Answer (2-Minute Version)

FP-Growth (Frequent Pattern Growth) is an association rule mining algorithm used to discover frequent itemsets efficiently. It improves upon Apriori by avoiding candidate itemset generation. The algorithm first builds a compressed FP-Tree from the transaction database

and then recursively mines frequent patterns from the tree. Because it requires fewer database scans and avoids generating large numbers of candidate itemsets, FP-Growth is generally faster and more scalable than Apriori for large datasets.